WorkTrace: Reconstructing and Comparing Work Progression in Coding Agents

Anonymous working manuscript

Annotated research draft — 26 October 2026

Abstract

Coding-agent evaluations summarize final task success and computational effort, leaving the organization of that effort difficult to compare. We present WorkTrace, a framework that relates tool operations to work targets, inferred code states, and earlier evidence. It combines an object-progression view of investigation with a code-version view of revision and verification, and aggregates traces into comparative patterns. We report current results from two complementary panels: 6,300 runs of three models under a common harness and 6,000 runs of one model under three harnesses. The saved analyses reveal two principal observations. First, models differ in verification placement: Gemini concentrates checks on the final inferred version, whereas Luna and Terra distribute fewer checks more evenly across versions. Second, harnesses exhibit different revisit structures despite similar exact-repeat rates. Pilot semantic labels contextualize these observations but do not establish causal utility. We describe the framework, compare systems, and distinguish the supported structural observations from claims about oververification, overexploration, and removable work that require further validation.

Draft status. All numerical results come from existing repository artifacts. No new agent experiments or human annotations were performed for this manuscript. Red text contains editorial improvement notes, separate from the proposed scientific claims.

REVISION 01. Regenerate the fact and graph outputs after the audited state/write fixes. Independent human validation is pending. The two abstract observations are provisional leads; do not promote them to verified mechanisms or "overthinking" without additional evidence.

1 Introduction

Language-model coding agents solve issues through repository inspection, implementation, and execution. Two agents may reach the same outcome through different processes: one may revise a small region and repeatedly test it, while another may inspect many targets before implementing a patch. Success, token consumption, and call counts expose the amount of work but leave its organization implicit.

Understanding that organization matters for comparing models and agent harnesses. A higher revisit rate can indicate incremental inspection, recovery from an unsuccessful approach, or reconstruction of context. A long verification phase can check additional properties or repeat evidence already obtained. Interpreting these behaviors requires connecting operations to the state and purpose of surrounding work.

Structural analysis offers a precedent. TRACE decomposes answer-bearing reasoning, reconstructs progression graphs, and connects induced patterns to utility tracing and a stopping case study (Zhang et al., 2026). Coding-agent traces introduce changing external states, heterogeneous observation scopes, and support scripts alongside implementation changes. A representation must preserve the relationship between investigation, code changes, and evidence.

We introduce WorkTrace for this comparative analysis. Its purpose is to explain how work unfolds. Candidate redundancy is one diagnostic within the framework, alongside exploration, refinement, recovery, and verification. We address three questions:

- **RQ1:** How can heterogeneous operations be represented as comparable work progressions?
- **RQ2:** What structural differences appear across models under a common harness and across harnesses with a fixed model?
- **RQ3:** Which conclusions require structural information, and what would establish their practical utility?

Our current observations concern *final-version verification concentration* and *harness-dependent revisit structure*. They motivate an explanatory study without presupposing that extended work is wasteful. They do not establish that particular checks or searches can safely be removed.

REVISION 02. State the explanation and decision enabled by each finding. The eventual contribution must exceed transferring TRACE's layout or observing that repetition differs from redundancy. Discover patterns on exploratory tasks and confirm the principal findings on fresh tasks.

2 Related Work

Structural trajectory analysis. TRACE studies answer progression in reasoning traces (Zhang et al., 2026). Software-agent trajectory analysis examines thought–action–result relationships and recurring sequences (Bouzenia and Pradel, 2025). WorkTrace extends this analytical direction by relating target visits to inferred implementation versions. Its intended contribution is a comparative account of changing work states and evidence, beyond an additional taxonomy alone.

Redundancy and efficiency. RedundancyBench evaluates redundant-step localization, categories, and supporting-evidence retrieval (Zhu et al., 2026). Its annotation and component-evaluation procedures offer useful validation precedents. AgentDiet compresses agent histories (Xiao et al., 2026); TRIM uses repair trajectories to minimize unnecessary final-patch edits (Mathai et al., 2026). Work organization, history compressibility, evidence novelty, patch minimality, and safe action removal require different evaluations.

Interaction. SWE-TaskFlow varies the interaction layer of verified tasks (Slinko et al., 2026). Such settings can test adaptation to new instructions. The present comparisons use the saved panels below and do not establish a controlled effect of user interaction.

REVISION 03. Complete claim-level positioning against these works. Credit adapted evidence/annotation procedures from RedundancyBench while retaining the software-work ontology. A redundancy-detector leaderboard would not establish this paper's intended explanatory contribution.

3 Benchmarking Agent Work

3.1 Comparison Panels

Cross-model panel. The JetBrains-derived analysis contains 6,300 runs: three models, 700 tasks, and three runs per task–model pair under mini-swe-agent. Identifiers follow the stored metadata: Gemini 3.7 Flash, GPT-5.6 Luna, and GPT-5.6 Terra. The fact table contains 188,609 operations; gold-patch metadata is available for 93.9% of runs. Table 1 reports saved benchmark results.

Cross-harness panel. Open-SWE-Traces v1.2 supplies Qwen3.8-27B runs under mini-swe-agent, OpenHands, and SWE-agent (NVIDIA, n.d.). The sample contains 2,000 task IDs attempted by all harnesses, with one selected run per task–harness pair: 6,000 runs and 370,992 operations. Tasks originate from SWE-rebench-V2 and Scale-SWE in the stored release. Table 2 reports the saved results. These panels answer conditional questions about models under one harness and harnesses for one model. Their task populations differ; they are not a fully crossed experiment and cannot be pooled to estimate model–harness interactions. Comparisons concern the released configurations rather than an isolated training or architectural effect.

3.2 Pilots and Statistical Units

The JetBrains semantic pilot contains 50 runs from 17 tasks and 1,432 operations: 789 Gemini, 315 Luna, and 328 Terra. One of 51 possible task–model combinations is absent because the sample was truncated to 50 runs. A second pilot contains 20 Open-SWE tasks across three harnesses, giving 60 runs and 4,018 operations. Every pilot operation has a stored primary label.

The primary annotator is recorded as DeepSeek-V4.1-Flash through the configured endpoint. GLM-5.3-Flash and three temperature-0.7 primary-model samples provide stability checks on 425 JetBrains items. Larger label shards exist, but the semantic results here use the explicitly defined pilots; partial scale labels are not pooled into a completed population study.

Saved intervals generally resample tasks, preserving associated runs and operations. The cross-model benchmark uses 300 bootstrap replicates; most v2 graph/pilot analyses use 1,000. We preserve the saved estimands and do not interpret overlapping intervals as equivalence. Return analysis uses task-clustered generalized estimating equations (GEE).

REVISION 04. Add paired differences with intervals for the main comparisons and harmonize run-mean versus pooled revisit rates. Identify source conditions/settings and missing-result categories. JetBrains metadata records 4,198 missing results: distinguish expected terminal omissions from genuinely missing observations before claiming complete logs.

4 The WorkTrace Framework

4.1 Stage 1: Operation Normalization

Operations record run/turn IDs, the agent message, invocation, observation, and available execution outcome. The parser splits shell chains for analysis while retaining the native tool call as the observed unit. It assigns read, search, edit, revert, run, probe, submit, or other kinds and identifies a main object. Mixed operations preserve constituent kinds.

Read objects commonly normalize to file paths; other objects can be commands or tool targets. These practical identifiers do not yet guarantee semantic equivalence across interfaces. Output normalization removes selected volatile fields for execution-like operations while preserving read content. Exact repetition requires normalized command and output-hash agreement.

4.2 Stage 2: Facts and Evidence-Linked Labels

For each operation, WorkTrace retrieves the latest same-object operation and the latest same-kind operation with matching usable output. Short or explicitly truncated outputs are excluded from output-candidate retrieval. Facts describe output agreement and whether the object or code may have changed since a candidate.

Semantic annotation separates *role* (e.g., explore, verify, revise), *relation* (unmatched, revisits, reproduces, uncertain), and *contribution* (distinct, non-distinct, uncertain). Labels cite operations and provide short rationales. For mixed operations, any distinct component makes the whole operation distinct.

The evidence validator restricts matched references to retrieved candidates and requires current and matched references for reproduces or `non_distinct`. An unmatched operation therefore cannot receive an accepted `non_distinct` label. Candidate-redundancy rates are reported under this restriction, rather than as a census of unnecessary work.

Model	Resolve (%)	Calls/run	Input tokens (M)	Exact repeats (%)	Object revisits (%)
Gemini 3.7 Flash	60.0	53.1	3.34	2.21 [2.08, 2.35]	21.1
GPT-5.6 Luna	50.0	17.6	0.25	0.02 [0.00, 0.03]	11.1
GPT-5.6 Terra	54.2	19.0	0.24	0.05 [0.03, 0.07]	9.4

Table 1: Cross-model benchmark: 2,100 runs per model and 700 matched tasks under mini-swe-agent. Exact-repeat and revisit rates are means of per-run rates. Brackets reproduce task-bootstrap 95% intervals for exact repeats; rounding does not imply a zero lower bound.

Harness	Resolve (%)	Calls/run	Assistant text (k chars)	Exact repeats (%)	Object revisits (%)
mini-swe-agent	57.1	51.8	110.6	0.20 [0.14, 0.28]	19.3 [18.9, 19.7]
OpenHands	51.0	66.2	149.2	0.18 [0.16, 0.21]	33.5 [33.0, 33.9]
SWE-agent	54.5	67.5	258.1	0.20 [0.17, 0.23]	30.9 [30.4, 31.3]

Table 2: Cross-harness benchmark: Qwen3.8-27B on the same 2,000 tasks. Exact repeats are run means; revisits are pooled operation rates, following the source table. Brackets are task-bootstrap 95% intervals. Characters are recorded proxies, not tokens or monetary costs.

REVISION 05. Audit retrieval recall and allow valid evidence outside the two candidates. The unmatched-distinct skip check is constrained by the label rules; it cannot independently establish usefulness. Keep rule assignments separate from LLM annotations and preserve uncertain evidence.

4.3 Stage 3: Two Progression Views

Object progression. Nodes represent targets and edges represent operations moving from the previous target to the current one. Immediate repeats form self-loops; later returns form back edges. A target can include purpose, but free-text purpose variation makes exact recurrence sparse. Existing analyses therefore use the object-only view as primary and retain purpose as supporting annotation.

For a return to target u at operation t, with preceding visit t', the detour count is

$$k_t = \left|\{u_j : t' < j < t\} \setminus \{u\}\right|. \quad (1)$$

It counts intervening distinct targets, not distinct hypotheses or alternative solutions.

Code-version progression. Nodes represent inferred repository states. Recognized writes create provisional versions; unknown writes can create uncertain versions. Recognized restorations and content observations can map work back to earlier nodes. Non-writing execution attaches as a verification self-loop, except for detected read-only inline inspections. Edges describe initial moves, revisions, revisions after failures, backtracks, and no-ops.

The implementation calls a revision after a failed check `correct`. This rule label describes temporal context; it does not certify a correct patch. Final resolved status comes from the dataset, while intermediate versions have no independent correctness oracle.

Our primary figures exclude writes confined to inferred support files from implementation versions. Support candidates use file history and final submission metadata. This is a retrospective descriptive view, not an online estimator. Every-write and known-write-only variants are retained for sensitivity analysis.

REVISION 06. Fix status-based false identity, branch switches, incomplete mutation paths, and unknown effects. Failed restores and matching clipped diffs must not establish state equality. Recompute all downstream summaries; separate hindsight-based support hints from facts available online.

4.4 Stage 4: Pattern Aggregation

Version graphs align by order and row: new versions advance horizontally and a new path after a backtrack advances vertically. Runs group by model and inferred version count. Node/edge weights record presence across runs; loops report mean verification operations. The saved figures prune components below 30% presence and display groups of at least 20 runs.

Figure 3 redraws saved four-version aggregates. Conditioning on version count controls one structural attribute but also selects on a trajectory outcome; these are illustrative slices, not unconditional model effects. Saved cross-harness aggregate version figures are not yet available.

REVISION 07. Test alignment/grouping/pruning sensitivity and report unpruned backtrack frequencies. A linear-looking aggregate can result from filtering or coordinate alignment. Validate each structural distinction before using it to explain system behavior.

5 Comparative Findings

5.1 Observation I: Verification Placement

The support-excluded reconstruction yields mean inferred version counts of 4.78 for Gemini, 2.61 for Luna, and 2.59 for Terra. Recognized backtracks occur in 3.7%, 0.2%, and 0.7% of runs. Recovered implementation trajectories are predominantly linear under these rules, which does not prove that an agent considered only one internal approach. Gemini has 3.20 [3.01, 3.40] verification operations on the final inferred version versus 0.71 [0.63, 0.80] per earlier version. Corresponding values are 0.39 versus 0.46 for Luna and 0.47 versus 0.54 for Terra (Figure 2). The final version receives 54.5% of verification for Gemini and approximately 32% for each GPT model in this selected population.

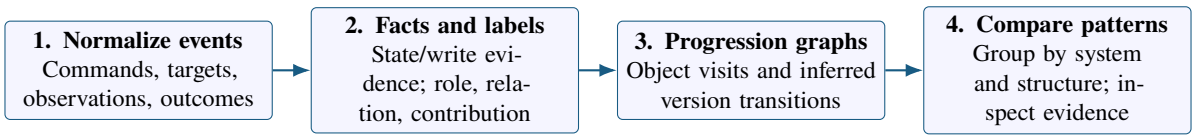


Figure 1: WorkTrace pipeline. The graph views share operation IDs, enabling investigation to be inspected alongside implementation and verification. Current patterns are coordinate-aligned aggregates, not learned graph clusters or causal models.

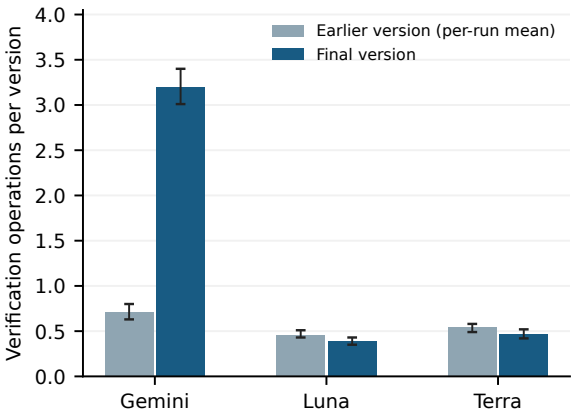


Figure 2: Verification placement in runs with at least two inferred versions: 1,722 Gemini, 1,514 Luna, and 1,459 Terra. Earlier-version values first average within each run. Bars reproduce saved task-bootstrap 95% intervals under the support-excluded reconstruction.

What comprises the final checks? Reconciliation compares each Gemini final-version check with earlier checks on that inferred version. Across 2,100 runs, 91.9% [90.8, 93.0] use a different command, 1.9% [1.5, 2.3] are identical command/output reruns, and 6.2% use the same command with different output. The first check is different-command by definition. Approximately 95% of multi-version runs have no identical rerun on the final version.

Rough command subtyping identifies suites, inline scripts, other test files, builds, and lint/type checks. This establishes heterogeneous command syntax, not necessarily distinct properties checked. In the 16-run Gemini pilot, 40 checks attach to final versions; 33 of 36 different-command checks are labeled distinct and three uncertain. One identical-rerun label contradicts the audited intervening facts, illustrating the need for independent semantic validation.

Interpretation. The representation localizes a difference that call counts cannot specify: Gemini's execution work accumulates before implementation and on the terminal inferred state, while Luna and Terra attach fewer checks throughout their version chains. This motivates a hypothesis about transitions from diagnostic checking to assurance. It does not yet establish expanded coverage, adequate confidence, or excessive verification.

REVISION 08. First phenomenon candidate: extract test/property scope and relevant-state dependencies, then separate diagnostic, change-validation, and additional-property checks. Account for length, version count, and unequal opportunity for final-state verification. Do not equate the final version with the first correct version or transfer TRACE's correctness-based interpretation without an oracle.

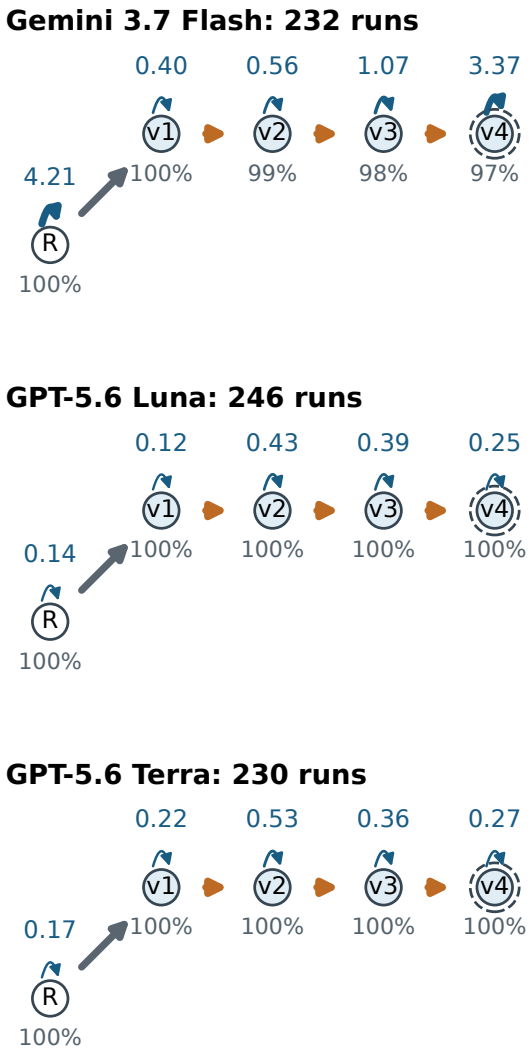


Figure 3: Saved four-version aggregates redrawn without changing results. Nodes are inferred versions; percentages below show presence, blue self-loops show mean verification operations, and dashed rings mark frequently final coordinates. Arrows show dominant retained forward moves; components below 30% presence are pruned. No intermediate correctness is marked.

5.2 Observation II: Harness Revisit Patterns

With Qwen fixed on matched tasks, exact-repeat run means are similar in magnitude: 0.20%, 0.18%, and 0.20% for mini-swe-agent, OpenHands, and SWE-agent. Pooled object revisits span 19.3% to 33.5%, and calls range from 51.8 to 67.5 (Table 2). These descriptive comparisons do not constitute an equivalence test.

Signature	Matches	Runs	ND–exact (pp)
Unchanged reread	12	6	+75.0 [16.7,95.0]
Immediate repeat	60	20	+13.3 [4.0,23.0]
Return after detour	153	34	+7.2 [−1.4,15.8]

Table 3: Pilot signatures. The final column subtracts exact-repeat share from non-distinct share within each signature. This is a descriptive label contrast, not an accuracy improvement over a detector. ND means non-distinct.

Reads, searches, and edits comprise 30.5%, 21.3%, and 12.0% of mini-swe-agent operations; 23.8%, 19.7%, and 23.4% for OpenHands; and 26.8%, 18.6%, and 18.4% for SWE-agent. Similar exact-repeat statistics thus coexist with different observed distributions of work.

Some revisit differences are representational. Structured editor views reuse file identifiers, whereas shell commands express separate ranges or search contexts. Current main-object extraction does not fully resolve this mismatch. The result is a difference in observed revisit structure; fragmented investigation, useful incremental inspection, and context reconstruction remain competing explanations.

The fully labeled 60-run Qwen pilot supplies an initial semantic slice. Non-distinct contribution among candidate-bearing operations is 5.1% [2.7, 7.7] for mini-swe-agent, 5.2% [2.8, 7.8] for OpenHands, and 9.7% [6.9, 13.3] for SWE-agent. These are pilot estimates, not semantic labels for the entire 6,000-run population; their differences need paired task analysis.

REVISION 09. Second phenomenon candidate: normalize file/span access, cross-tool representations, call granularity, and output completeness. Classify residual returns by evidence acquired. Add cross-harness progression figures and paired contrasts; a revisit count cannot establish memory loss or overexploration.

Outlier sensitivity. One mini-swe-agent trace issues the same Python probe 164 times with empty output. Excluding its task changes that harness's pooled exact-repeat rate from 0.32% to 0.17%, and its run mean from 0.20% to 0.16%. No other sampled run has more than eight exact repeats. This is a useful failure case, not a broad population pattern.

5.3 Supporting Return Analysis

The JetBrains pilot has 225 candidate-bearing operations, of which 51 are non-distinct (22.7% [16.7, 29.6]). There are 21 candidate-redundant operations, defined as reproduces and `non_distinct`, among 1,432 operations. Their pooled rate is 1.5%; the mean of per-run rates is 1.1%. Neither estimates all unnecessary work. Of 153 returns, two uncertain contributions are excluded from GEE analysis. The remaining 151 contain 32 non-distinct events in 17 task clusters. Adjusted odds ratios relative to one detour are 32.7 [2.3, 456] for two to three detours and 14.5 [1.1, 193] for four or more, controlling relative position, log run length, and model identity. The pattern mainly occurs in long runs and has wide uncertainty.

Predictor	Predicted	Precision	Recall
Exact repeat	412	5.6%	33.3%
Object revisit	4,599	0.6%	39.1%
Candidate redundant	536	3.5%	27.5%

Table 4: Comparison against CodeTraceBench's unuseful labels. WorkTrace uses the lower uncertain bound. The different target construct prevents direct interpretation as accuracy on WorkTrace contribution labels.

REVISION 10. Replicate return-after-detour findings before treating them as a general phenomenon. There are approximately five events per slope parameter and 17 clusters. The reread signature also uses facts shown to the annotator, so its strong label agreement is not independent validation of the construct.

6 Validation and Diagnostic Value

6.1 Annotation Stability

The secondary annotator labels all 225 matched pilot operations and 200 sampled unmatched operations. Matched contribution agreement is 90.2% ($\kappa = 0.70$); matched relation agreement is 84.0% ($\kappa = 0.50$). Candidate redundancy has $\kappa = 0.83$ across 425 items. Primary-model self-consistency gives contribution $\kappa = 0.88$–0.90 on matched operations.

These checks establish selected forms of stability, not human accuracy. The exported 200-item human packet has no adjudicated contribution labels. Both automatic annotators receive the same restricted candidates and rule facts, so they can share systematic errors.

6.2 External Construct Check

The stored CodeTraceBench comparison evaluates 484 aligned trajectories, 17,317 steps, and 69 human-annotated unuseful steps (NJU-LINK, n.d.). Table 4 preserves the unfavorable comparison with exact repeats. Only 32 of 69 human-labeled steps have a WorkTrace candidate, limiting recall under the present evidence rule. Construct differences limit interpretation but do not turn the result into positive validation.

REVISION 11. Build an independent reference across both panels: unmatched work, incomplete logs, relevant/irrelevant edits, and cross-tool equivalents. Report fact accuracy, retrieval recall, semantic accuracy, uncertainty, and how corrections change structural findings. Adapt RedundancyBench's guide/adjudication/component procedures while retaining difficult and redundancy-free cases.

6.3 What the Framework Currently Adds

The linked views locate effort within a progression. They distinguish Gemini's final-state checking from implementation churn and reveal Qwen harness revisit differences that exact repeats obscure. These are concrete diagnostic uses; they do not establish that semantic labels are accurate or that graphs are necessary for every conclusion.

A stronger utility evaluation would compare diagnoses based on counts, sequences, state-only summaries, and the full framework. Removing version context should challenge verification-placement explanations; removing target/scope normalization should challenge harness-revisit

explanations. Any predictive comparison should split by task and avoid predicting labels from the facts that define those same labels.

REVISION 12. Add a direct framework-utility test: blinded diagnoses with simple summaries versus WorkTrace, or held-out prediction of an independent outcome beyond length, cost, action histograms, and state-only features. Report what changes, what remains unexplained, and counterexamples.

7 Interpreting Excessive Work

7.1 Identifying Excessive Verification

Verification volume, command diversity, and evidence novelty are separate. The same tests on a relevant new state can establish a new claim; different commands can check the same properties on unchanged code. Stochastic reruns can add confidence even with matching visible output. A thick loop is therefore not an oververification label.

An extension should link each check to relevant state, selected tests/assertions, configuration, and prior evidence. Repetition candidates can then be separated from demonstrated excess. Oververification requires insufficient benefit for its cost under an explicit decision criterion.

7.2 Identifying Excessive Exploration

Exploration should be tied to unresolved questions or causes. Command changes and file visits do not directly identify strategy branches. Inspecting an ultimately irrelevant file can eliminate a plausible cause; returning to a settled question without new grounds is a stronger unproductive-work candidate. Current return measurements locate such cases without deciding their utility.

Prospective utility test. A next study could vary continued checking or investigation at selected prefixes, measuring subsequent behavior, success, and total cost including analysis overhead. Action deletion, observation reuse, history compression, and termination are different interventions. A patch passing before and after checks does not prove they were useless, and deleting events while holding a recorded suffix fixed does not reproduce an adaptive agent counterfactual.

REVISION 13. No intervention result is currently available. Choose a utility experiment only if it tests the main explanation. Predefine the success/cost tradeoff and an adequate sample size; absence of a significant success difference does not establish preserved performance.

8 Limitations

Measurement reliability. Reproducible failure cases affect status-based identity, branch changes, unknown effects, multi-target writes, failed restores, and clipped observations. These concern the premises of state and contribution analysis. All quoted numbers are saved pre-repair results; their real-trajectory impact needs regeneration.

Construct validity. Command syntax does not establish coverage or usefulness. Labels are candidate-constrained and not independently human-validated. Support-file inference uses final submissions. Purpose strings do not reliably identify hypotheses, and version chains do not expose hidden reasoning.

Comparative scope. Task populations differ across panels and model–harness coverage is incomplete. Tool granularity, settings, stopping rules, missing exit codes, and incomplete observations can confound comparisons. The semantic pilots are small and have already informed exploration.

Utility and generalization. There is no completed demonstration of safe effort reduction. Operation rates are not wasted-time or monetary estimates. New user requirements and interactive development are outside the current comparison. Later extensions should preserve instruction boundaries and distinguish adaptation from reconstruction.

9 Conclusion

WorkTrace relates operations to targets, inferred versions, and evidence to analyze coding-agent work beyond final success and aggregate effort. Saved results expose differences in verification placement across models and revisit structure across harnesses. They motivate a structural account of exploration, revision, and verification while leaving mechanisms and utility open. Independent validation, common measurement, and fresh-task confirmation are the next steps toward a reliable comparative explanation.

REVISION 14. Rewrite the abstract, introduction, and conclusion around confirmed findings after revision. Keep the framework and cross-system discoveries central. Repetition versus redundancy is a supporting measurement distinction, not the final paper's headline.

References

Xinliang Frederick Zhang, Anhad Mohananey, Alexandra Chronopoulou, Pinelopi Papalampidi, Somit Gupta, Tsendsuren Munkhdalai, Lu Wang, and Shyam Upadhyay. 2026. Do LLMs Really Need 10+ Thoughts for "Find the Time 1000 Days Later"? Towards Structural Understanding of LLM Overthinking. In *Proceedings of ACL*, pages 17005–17030. https://aclanthology.org/2026.acl-long.773/.

Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. https://www.software-lab.org/publications/ase2025_trajectories.pdf.

Liang Zhu, Bo Yang, Zhinuo Zhou, Jiachen Liang, Jiahao Guo, Yiyang Yin, Xiongwei Han, and Minyang Hu. 2026. Redundant or Necessary? A Benchmark for Detecting Redundant Steps in Agent Trajectories. arXiv:2605.29893v2. https://arxiv.org/abs/2605.29893v2.

Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. arXiv:2509.23586v2. https://arxiv.org/abs/2509.23586v2.

Alex Mathai, Shobini Iyer, Aleksandr Nogikh, Petros Maniatis, Franjo Ivančić, Junfeng Yang, and Baishakhi Ray. 2026. TRIM: Reducing AI-Generated CodeSlop via Agent Trajectory Minimization. arXiv:2607.18161. https://arxiv.org/abs/2607.18161.

Igor Slinko, Yaroslav Golubev, and Sergey Titov. 2026. Opinion: Coding-Agent Benchmarks Should Match Their Users' Task Flows. arXiv:2610.09633v2. https://arxiv.org/abs/2610.09633v2.

NVIDIA. n.d. Open-SWE-Traces, v1.2 dataset release. https://huggingface.co/datasets/nvidia/Open-SWE-Traces.

NJU-LINK. n.d. CodeTraceBench, dataset release. https://huggingface.co/datasets/NJU-LINK/CodeTraceBench.

A Artifact Provenance

Paths below are relative to the repository root. The draft uses saved artifacts rather than claiming an end-to-end rerun; raw logs, issues, and some caches are absent from this checkout.

Benchmark tables. Table 1 uses `results/v2/benchmark/table1_by_model.csv`. Table 2 uses `results/v2/task5/table1_by_harness.csv`. Outlier sensitivity uses `results/v2/task5/exact_repeat_with_without_outlier.csv`.

Version patterns. Figure 2 uses `results/v2/viewb/support_no_version/measures_by_model.csv`. Figure 3 redraws the four-version entries of `results/v2/viewb/support_no_version/aggregates.json`. The plotting script performs no new graph inference or model calls.

Final-check reconciliation. Command classes use `results/v2/reconcile/scale_summary.csv`. Population definitions matter: the graph summary gives 3.20 final checks for multi-version Gemini runs; reconciliation gives 3.17. Six traces end at root: the graph summary includes checks attached there across the trajectory, while reconciliation assigns zero to those runs.

Pilot labels and signatures. Contribution counts use `results/v2/task2/contribution_tables.csv`; run means use `results/v2/task2/run_measures.csv`; signature contrasts use `results/v2/task2/signatures_ci.csv`. There are 51 non-distinct labels and 21 candidate-redundant labels among 1,432 operations. Mean-of-run and pooled rates are explicitly distinguished.

Regression and validation. Return coefficients use `results/v2/task3/coefficients.csv`. Agreement uses `results/v2/task4/agreement.csv`; the external comparison uses `results/v2/task4/codetracebench/prf.csv`. Qwen pilot values use `results/v2/qwen/pilot/pilot_summary.csv`. The exported human packet is `annotation/pilot.csv`.

Dataset revisions. JetBrains records revision `a5ec6ad84287`; Open-SWE records `a85dc97891bd`. Full hashes appear in the manuscript manifest. These provenance strings do not establish that the draft's analyses were regenerated.

REVISION 15. Release a machine-readable manifest mapping each figure/table to source condition, dataset/parser revision, prompt, settings, labels, and script. Pin dependencies and preserve complete evidence inputs. Shortened viewer displays cannot reconstruct full original observations.

B Priority Revision Experiments

Measurement repair. Turn the seven audit cases into passing regression checks. Recompute facts, candidates, graphs, and labels whose inputs change. Measure the effect of repairs on both main observations.

Independent reference. Calibrate the guide on development examples, then freeze a task-separated sample across models/harnesses. Preserve independent judgments before adjudication. Include mixed operations, cross-tool equivalents, changed-state relevance, and incomplete context.

Comparative confirmation. Compute task-paired contrasts under alternate target normalization and graph thresholds. Select phenomena using exploratory tasks and confirm their practical magnitude on fresh tasks. A crossed experiment is needed for interaction claims.

Verification scope. Recover test selection, assertions, configuration, and state dependencies. Separate new syntax from new coverage and examine checks that uncover defects or lead to revisions. Checks can be useful without modifying code.

Exploration semantics. Link returns to stated questions and available evidence. Distinguish incremental inspection and justified recovery from revisiting settled questions without new grounds. Do not infer hidden beliefs from graph shape.

Optional interactive boundary test. Use suitable matched Single/Concat/Split traces if available, retaining requirement boundaries. This tests progressive disclosure rather than live human collaboration. Add reactive-user data only if the eventual explanation needs it.

REVISION 16. One-week foundation: days 1–2 repair/version measurements and the guide; days 3–4 conduct the human pilot if annotators are available; days 5–6 rebuild comparisons and challenge explanations; day 7 consolidate figures and define fresh-task confirmation. Strengthen the evidence rather than merely increasing label volume.